

Arena Strike 3D

Poora Source Code - 4 Languages/Technologies

Ye game ek single self-contained .html file mein bana hai, jisme 4 cheezein use hui hain:

- 1. HTML** - Page ka structure: buttons, HUD panels, health bar, overlay screens, mobile control buttons.
- 2. CSS** - Styling: colors, health bar animation, crosshair, minimap box, mobile joystick design, responsive layout.
- 3. JavaScript** - Asli game logic: player movement, shooting, enemy AI, loot system, wave spawning, touch controls, collision detection, HUD updates.
- 4. Three.js** (JavaScript library) - 3D graphics engine jo JavaScript ke andar hi use hota hai: camera, lighting, 3D models (walls, enemies, loot), shadows, raycasting for shooting.

Neeche har section mein poora code hai, language-wise alag kiya hua.

1. HTML - Page Structure

Ye HTML markup hai: head, meta tags, HUD elements, buttons, overlay screen aur mobile touch control divs. (CSS agle section mein hai, JavaScript ke andar hi Three.js bhi use hota hai jo section 4 mein hai.)

```
<!DOCTYPE html>
<html lang="hi">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1, user-
scalable=no, viewport-fit=cover">
<title>Arena Strike 3D</title>
<style>
/* ... CSS shown in the CSS section ... */
</style>
</head>
<body>
<div id="hud">
<div id="topbar">
<div class="panel" id="scorePanel">Score: <span id="scoreVal">0</span></div>
<div class="panel" id="wavePanel">Wave <span id="waveVal">1</span></div>
</div>
<div id="minimapWrap"><canvas id="minimap" width="130" height="130"></canvas></div>
<div id="bottombar">
<div class="panel" id="healthWrap">
<div id="healthLabel">HEALTH</div>
<div id="healthBarOuter"><div id="healthBarInner"></div></div>
</div>
<div class="panel" id="ammoPanel">
<span id="weaponName">Pistol</span><br>
<span id="ammoVal">30</span> / <span id="reserveVal">60</span>
<small id="reloadHint">R to reload</small>
</div>
</div>
</div>
<div id="crosshair"></div>
<div id="hitFlash"></div>
<div id="muzzle"></div>
<div id="pickupToast"></div>
<div id="touchControls">
<div id="lookZone"></div>
<div id="joyBase"><div id="joyStick"></div></div>
<div id="reloadBtnMobile">RELOAD</div>
<div id="fireBtn">FIRE</div>
</div>
<div id="overlay">
<h1>ARENA STRIKE 3D</h1>
<p>Arena mein loot uthao – ammo, health, naye weapons – aur dushmano ki waves ko khatam karte
jao.</p>
<p class="hint" id="desktopHint">
<b>W A S D</b><b> move &nbsp;&nbsp;&nbsp;</b><b> Mouse</b><b> look &nbsp;&nbsp;&</b><b> Click</b><b> shoot &nbsp;&nbsp;&</b>
<b> R</b><b> reload &nbsp;&nbsp;&</b><b> Shift</b><b> sprint</b>
</p>
<p class="hint" id="mobileHint" style="display:none;">Left side: joystick se move &nbsp;&nbsp;&;&nbsp;&nbsp;&
Right side: drag se look &nbsp;&nbsp;&;&nbsp;&nbsp;& FIRE button se shoot</p>
<button class="btn" id="startBtn">Tap / Click to Start</button>
<div class="stats" id="finalStats" style="display:none;"></div>
</div>
</body>
</html>
```

2. CSS - Styling

Ye poora CSS hai jo game ke andar style tag mein likha hai - colors, HUD design, health bar, crosshair, minimap box, aur mobile joystick/buttons ka look.

```
:root{
  --accent:#ffb020;
  --danger:#ff4d4d;
  --hud-bg: rgba(10,12,16,0.55);
  --hud-fg: #f2f2f2;
}
html,body{margin:0;padding:0;overflow:hidden;background:#000;height:100%;width:100%;font-family:'Segoe UI',Arial,sans-serif;touch-action:none;-webkit-user-select:none;user-select:none;}
canvas{display:block;}

#crosshair{
  position:fixed;top:50%;left:50%;transform:translate(-50%,-50%);
  width:22px;height:22px;pointer-events:none;z-index:10;
}
#crosshair::before,#crosshair::after{ content:""; position:absolute; background:var(--accent); }
#crosshair::before{ left:50%; top:0; width:2px; height:100%; transform:translateX(-1px); }
#crosshair::after{ top:50%; left:0; height:2px; width:100%; transform:translateY(-1px); }

#hud{ position:fixed; inset:0; pointer-events:none; z-index:9; color:var(--hud-fg); }
#topbar{
  position:absolute; top:12px; left:12px; right:12px;
  display:flex; justify-content:space-between; align-items:flex-start; gap:8px;
}
.panel{
  background:var(--hud-bg); border-radius:10px; padding:8px 14px;
  backdrop-filter:blur(4px); font-size:13px; letter-spacing:0.3px;
}
#scorePanel{ font-size:15px; font-weight:600; }
#bottombar{
  position:absolute; bottom:12px; left:12px; right:12px;
  display:flex; justify-content:space-between; align-items:flex-end; gap:8px;
}
#healthWrap{ width:200px; max-width:42vw; }
#healthLabel{ font-size:11px; opacity:0.85; margin-bottom:4px; }
#healthBarOuter{
  width:100%; height:13px; background:rgba(255,255,255,0.15);
  border-radius:8px; overflow:hidden; border:1px solid rgba(255,255,255,0.25);
}
#healthBarInner{
  height:100%; width:100%; background:linear-gradient(90deg,#ff4d4d,#ff8a4d);
  transition:width 0.15s ease;
}
#ammoPanel{ font-size:15px; font-weight:600; text-align:right; }
#ammoPanel small{ display:block; font-size:11px; font-weight:400; opacity:0.8; }
#weaponName{ color:var(--accent); }

#minimapWrap{
  position:absolute; top:60px; right:12px; width:130px; height:130px;
  background:var(--hud-bg); border-radius:10px; border:1px solid rgba(255,255,255,0.2);
  overflow:hidden;
}

#hitFlash{ position:fixed; inset:0; background:rgba(255,0,0,0.25); opacity:0; pointer-events:none;
  z-index:8; transition:opacity 0.1s ease; }
#muzzle{
  position:fixed; width:70px; height:70px; left:50%; top:58%;
  transform:translate(-50%,-50%); pointer-events:none; z-index:7;
  background:radial-gradient(circle, rgba(255,200,80,0.9) 0%, rgba(255,150,0,0.4) 40%, transparent
    70%);
  opacity:0; border-radius:50%;
}
#pickupToast{
  position:fixed; top:40%; left:50%; transform:translate(-50%,-50%);
  color:var(--accent); font-weight:700; font-size:18px; opacity:0; pointer-events:none;
  z-index:11; text-shadow:0 2px 6px rgba(0,0,0,0.8); transition:opacity 0.25s ease, transform
    0.25s ease;
}

#overlay{
  position:fixed; inset:0; background:rgba(5,6,10,0.9);
  display:flex; flex-direction:column; align-items:center; justify-content:center;
```

```

    z-index:30; text-align:center; color:#fff; padding:20px; box-sizing:border-box;
}
#overlay h1{ font-size:clamp(26px,6vw,42px); margin:0 0 8px; color:var(--accent); }
#overlay p{ max-width:480px; line-height:1.6; opacity:0.9; font-size:14px; }
#overlay .btn{
    margin-top:20px; padding:14px 34px; font-size:16px; font-weight:700;
    background:var(--accent); color:#1a1a1a; border:none; border-radius:10px;
    cursor:pointer; letter-spacing:0.5px;
}
#overlay .btn:hover{ filter:brightness(1.08); }
#overlay .stats{ margin-top:16px; font-size:15px; color:var(--accent); font-weight:600; }
#overlay.hidden{ display:none; }
.hint{ font-size:12px; opacity:0.65; margin-top:12px; }
kbd{ background:rgba(255,255,255,0.12); padding:2px 7px; border-radius:5px; font-size:11px; }

/* Mobile touch controls */
#touchControls{ position:fixed; inset:0; z-index:15; display:none; pointer-events:none; }
#touchControls.active{ display:block; }
#lookZone{ position:absolute; right:0; top:0; width:60%; height:100%; pointer-events:auto; }
#joyBase{
    position:absolute; left:26px; bottom:26px; width:110px; height:110px;
    border-radius:50%; background:rgba(255,255,255,0.12); border:2px solid rgba(255,255,255,0.3);
    pointer-events:auto;
}
#joyStick{
    position:absolute; left:50%; top:50%; width:48px; height:48px; margin:-24px;
    border-radius:50%; background:rgba(255,176,32,0.75);
}
#fireBtn{
    position:absolute; right:26px; bottom:34px; width:78px; height:78px; border-radius:50%;
    background:rgba(255,77,77,0.75); border:2px solid rgba(255,255,255,0.4);
    color:#fff; font-weight:700; font-size:13px; display:flex; align-items:center; justify-
    content:center;
    pointer-events:auto;
}
#reloadBtnMobile{
    position:absolute; right:118px; bottom:46px; width:56px; height:56px; border-radius:50%;
    background:rgba(255,255,255,0.18); border:2px solid rgba(255,255,255,0.35);
    color:#fff; font-weight:700; font-size:11px; display:flex; align-items:center; justify-
    content:center;
    pointer-events:auto;
}
}

```

3 & 4. JavaScript + Three.js - Game Logic & 3D Engine

Ye poora JavaScript code hai jo game chalata hai. Three.js library (THREE.Scene, THREE.Camera, THREE.Mesh, THREE.Raycaster, etc.) isi JavaScript code ke andar use hoti hai 3D graphics, lighting, shadows aur shooting ke liye. Neeche dono ek saath hain kyunki Three.js JavaScript ke through hi call hoti hai.

```
(function(){
  const isTouch = ('ontouchstart' in window) || navigator.maxTouchPoints > 0;
  document.getElementById('mobileHint').style.display = isTouch ? 'block' : 'none';
  document.getElementById('desktopHint').style.display = isTouch ? 'none' : 'block';

  // ----- basic setup -----
  const scene = new THREE.Scene();
  scene.background = new THREE.Color(0x0b1622);
  scene.fog = new THREE.Fog(0x0b1622, 20, 75);

  const camera = new THREE.PerspectiveCamera(75, window.innerWidth/window.innerHeight, 0.1, 1000);
  camera.position.set(0, 1.7, 0);

  const renderer = new THREE.WebGLRenderer({antialias:true});
  renderer.setSize(window.innerWidth, window.innerHeight);
  renderer.setPixelRatio(Math.min(window.devicePixelRatio,2));
  renderer.shadowMap.enabled = true;
  document.body.appendChild(renderer.domElement);

  window.addEventListener('resize', ()=>{
    camera.aspect = window.innerWidth/window.innerHeight;
    camera.updateProjectionMatrix();
    renderer.setSize(window.innerWidth, window.innerHeight);
  });

  // ----- lights -----
  scene.add(new THREE.AmbientLight(0x8899aa, 0.7));
  const sun = new THREE.DirectionalLight(0xffffff, 0.9);
  sun.position.set(30,40,10);
  sun.castShadow = true;
  sun.shadow.mapSize.set(1024,1024);
  sun.shadow.camera.left=-40; sun.shadow.camera.right=40;
  sun.shadow.camera.top=40; sun.shadow.camera.bottom=-40;
  scene.add(sun);

  // ----- arena -----
  const ARENA = 34;
  const groundGeo = new THREE.PlaneGeometry(ARENA*2, ARENA*2, 10, 10);
  const groundMat = new THREE.MeshStandardMaterial({color:0x2c3e40, roughness:0.95});
  const ground = new THREE.Mesh(groundGeo, groundMat);
  ground.rotation.x = -Math.PI/2;
  ground.receiveShadow = true;
  scene.add(ground);

  const wallMat = new THREE.MeshStandardMaterial({color:0x1a2530, roughness:0.9});
  function addWall(x,z,w,d,h){
    const wall = new THREE.Mesh(new THREE.BoxGeometry(w,h,d), wallMat);
    wall.position.set(x,h/2,z);
    wall.castShadow = true; wall.receiveShadow = true;
    scene.add(wall);
  }
  addWall(0,-ARENA,ARENA*2,1,6);
  addWall(0,ARENA,ARENA*2,1,6);
  addWall(-ARENA,0,1,ARENA*2,6);
  addWall(ARENA,0,1,ARENA*2,6);

  const coverMat = new THREE.MeshStandardMaterial({color:0x4a5a52, roughness:0.85});
  const obstacles = [];
  function addBox(x,z,sx,sy,sz){
    const b = new THREE.Mesh(new THREE.BoxGeometry(sx,sy,sz), coverMat);
    b.position.set(x, sy/2, z);
    b.castShadow = true; b.receiveShadow = true;
    scene.add(b);
    obstacles.push({x:x, z:z, hw:sx/2, hd:sz/2});
  }
  const layout = [
    [8,8,4,2.2,4],[8,8,3,2.6,3],[8,-8,4,2,4],[-8,-8,3,2.4,5],
    [0,14,6,2,2],[0,-14,6,2,2],[14,0,2,2,6],[-14,0,2,2,6],
    [4,0,2,1.6,2],[-5,3,2,1.8,2],[18,18,3,2,3],[-18,-18,3,2,3]
  ];
});
```

```

layout.forEach(a=>addBox(...a));

// ----- weapons -----
const WEAPONS = {
  Pistol: {name:'Pistol', damage:34, magSize:30, reserveMax:90, cooldown:180, spread:0.01,
    color:0xffffffff},
  SMG: {name:'SMG', damage:18, magSize:40, reserveMax:160,cooldown:85, spread:0.03,
    color:0x3fa9f5},
  Shotgun: {name:'Shotgun', damage:16, magSize:8, reserveMax:32, pellets:6, cooldown:600,
    spread:0.09, color:0xf5a623},
  Sniper: {name:'Sniper', damage:100,magSize:5, reserveMax:20, cooldown:950, spread:0.002,
    color:0xd0021b}
};
let currentWeaponKey = 'Pistol';
let ammo = WEAPONS.Pistol.magSize;
let reserve = { Pistol:90, SMG:0, Shotgun:0, Sniper:0 };
let reloading = false;
let lastShotTime = 0;

// ----- player state -----
const player = { height:1.7, speed:5.2, sprintMult:1.6, health:100, maxHealth:100,
  pos:new THREE.Vector3(0,0,0), yaw:0, pitch:0, alive:true };

let score = 0, wave = 1, kills = 0, killsNeededForWave = 5;

// ----- pointer lock (desktop) / tap-start (mobile) -----
const canvas = renderer.domElement;
let gameActive = false;
const overlay = document.getElementById('overlay');
const startBtn = document.getElementById('startBtn');
const touchControls = document.getElementById('touchControls');

function beginGame(){
  if(isTouch){
    gameActive = true;
    overlay.classList.add('hidden');
    touchControls.classList.add('active');
  } else {
    canvas.requestPointerLock();
  }
}
startBtn.addEventListener('click', beginGame);

document.addEventListener('pointerlockchange', ()=>{
  if(isTouch) return;
  gameActive = (document.pointerLockElement === canvas);
  overlay.classList.toggle('hidden', gameActive);
  if(!gameActive && player.alive){
    overlay.classList.remove('hidden');
    document.getElementById('finalStats').style.display='none';
    startBtn.style.display='inline-block';
    startBtn.textContent = 'Resume (Click)';
  }
});

document.addEventListener('mousemove', (e)=>{
  if(!gameActive || isTouch) return;
  const sens = 0.0022;
  player.yaw -= e.movementX * sens;
  player.pitch -= e.movementY * sens;
  player.pitch = Math.max(-Math.PI/2+0.05, Math.min(Math.PI/2-0.05, player.pitch));
});

// ----- keyboard (desktop) -----
const keys = {};
window.addEventListener('keydown', e=>{
  keys[e.code]=true;
  if(e.code==='KeyR') tryReload();
  if(e.code==='Digit1') switchWeapon('Pistol');
  if(e.code==='Digit2') switchWeapon('SMG');
  if(e.code==='Digit3') switchWeapon('Shotgun');
  if(e.code==='Digit4') switchWeapon('Sniper');
});
window.addEventListener('keyup', e=> keys[e.code]=false);

// ----- mobile joystick -----
let joyVec={x:0,y:0};
const joyBase = document.getElementById('joyBase');
const joyStick = document.getElementById('joyStick');

```

```

let joyTouchId = null, joyOrigin={x:0,y:0};

joyBase.addEventListener('touchstart', e=>{
  e.preventDefault();
  const t = e.changedTouches[0];
  joyTouchId = t.identifier;
  const rect = joyBase.getBoundingClientRect();
  joyOrigin = {x: rect.left+rect.width/2, y: rect.top+rect.height/2};
}, {passive:false});

window.addEventListener('touchmove', e=>{
  for(const t of e.changedTouches){
    if(t.identifier===joyTouchId){
      let dx = t.clientX - joyOrigin.x, dy = t.clientY - joyOrigin.y;
      const max = 42;
      const dist = Math.min(Math.hypot(dx,dy), max);
      const ang = Math.atan2(dy,dx);
      dx = Math.cos(ang)*dist; dy = Math.sin(ang)*dist;
      joyStick.style.transform = `translate(${dx}px,${dy}px)`;
      joyVec = {x: dx/max, y: dy/max};
    }
  }
}, {passive:false});

window.addEventListener('touchend', e=>{
  for(const t of e.changedTouches){
    if(t.identifier===joyTouchId){
      joyTouchId=null; joyVec={x:0,y:0};
      joyStick.style.transform = 'translate(0,0)';
    }
  }
});

// ----- mobile look (drag on right zone) -----
const lookZone = document.getElementById('lookZone');
let lookTouchId=null, lastLookX=0, lastLookY=0;
lookZone.addEventListener('touchstart', e=>{
  e.preventDefault();
  const t = e.changedTouches[0];
  lookTouchId = t.identifier;
  lastLookX = t.clientX; lastLookY = t.clientY;
}, {passive:false});
lookZone.addEventListener('touchmove', e=>{
  e.preventDefault();
  for(const t of e.changedTouches){
    if(t.identifier===lookTouchId){
      const dx = t.clientX-lastLookX, dy = t.clientY-lastLookY;
      lastLookX=t.clientX; lastLookY=t.clientY;
      const sens = 0.0032;
      player.yaw -= dx*sens;
      player.pitch -= dy*sens;
      player.pitch = Math.max(-Math.PI/2+0.05, Math.min(Math.PI/2-0.05, player.pitch));
    }
  }
}, {passive:false});
lookZone.addEventListener('touchend', e=>{
  for(const t of e.changedTouches) if(t.identifier===lookTouchId) lookTouchId=null;
});

// ----- fire / reload buttons (mobile) -----
document.getElementById('fireBtn').addEventListener('touchstart', e=>{
  e.preventDefault(); shoot();
}, {passive:false});
document.getElementById('reloadBtnMobile').addEventListener('touchstart', e=>{
  e.preventDefault(); tryReload();
}, {passive:false});

// ----- shooting (desktop click) -----
const raycaster = new THREE.Raycaster();
const muzzleEl = document.getElementById('muzzle');
const hitFlashEl = document.getElementById('hitFlash');

canvas.addEventListener('mousedown', (e)=>{
  if(!gameActive || isTouch || e.button!==0) return;
  shoot();
});

function switchWeapon(key){
  if(currentWeaponKey===key || reloading) return;

```

```

    currentWeaponKey = key;
    ammo = WEAPONS[key].magSize;
    updateHUD();
}

function tryReload(){
    if(reloading || !player.alive) return;
    const w = WEAPONS[currentWeaponKey];
    if(ammo===w.magSize || reserve[currentWeaponKey]<=0) return;
    reloading = true;
    document.getElementById('reloadHint').textContent = 'Reloading...';
    setTimeout(()=>{
        const need = w.magSize - ammo;
        const take = Math.min(need, reserve[currentWeaponKey]);
        ammo += take;
        reserve[currentWeaponKey] -= take;
        reloading = false;
        updateHUD();
    }, 1000);
}

function shoot(){
    if(!gameActive || !player.alive || reloading) return;
    const w = WEAPONS[currentWeaponKey];
    const now = performance.now();
    if(now - lastShotTime < w.cooldown) return;
    if(ammo<=0){ tryReload(); return; }
    lastShotTime = now;
    ammo--;
    updateHUD();
    muzzleEl.style.opacity = 1;
    setTimeout(()=>muzzleEl.style.opacity=0, 60);

    const pellets = w.pellets || 1;
    const meshes = enemies.map(en=>en.mesh);
    for(let p=0;p<pellets;p++){
        const sx = (Math.random()-0.5)*w.spread*2;
        const sy = (Math.random()-0.5)*w.spread*2;
        raycaster.setFromCamera({x:sx,y:sy}, camera);
        const hits = raycaster.intersectObjects(meshes, true);
        if(hits.length>0){
            let obj = hits[0].object;
            while(obj.parent && !obj.userData.enemyRef) obj = obj.parent;
            const enemy = obj.userData.enemyRef;
            if(enemy) damageEnemy(enemy, w.damage);
        }
    }
}

// ----- enemies -----
const enemies = [];
const enemyGeo = new THREE.CapsuleGeometry(0.45, 1.1, 4, 8);
function spawnEnemy(){
    const mat = new THREE.MeshStandardMaterial({color:0xc0392b, roughness:0.7});
    const mesh = new THREE.Mesh(enemyGeo, mat);
    mesh.castShadow = true;
    let x,z;
    do{
        const ang = Math.random()*Math.PI*2;
        const r = 15 + Math.random()*15;
        x = Math.cos(ang)*r; z = Math.sin(ang)*r;
    } while(Math.abs(x)>ARENA-2 || Math.abs(z)>ARENA-2);
    mesh.position.set(x, 1.05, z);
    scene.add(mesh);
    const enemy = { mesh, health:60, speed: 1.3+Math.random()*0.6+wave*0.06, alive:true, attackCd:0
    };
    mesh.userData.enemyRef = enemy;
    enemies.push(enemy);
}

function damageEnemy(enemy, dmg){
    if(!enemy.alive) return;
    enemy.health -= dmg;
    enemy.mesh.material.emissive = new THREE.Color(0x330000);
    setTimeout(()=>{ if(enemy.mesh.material) enemy.mesh.material.emissive = new
    THREE.Color(0x000000); },80);
    if(enemy.health<=0){
        enemy.alive = false;
        scene.remove(enemy.mesh);
    }
}

```

```

    score += 100;
    kills++;
    updateHUD();
    checkWave();
}
}

function checkWave(){
    if(kills >= killsNeededForWave){
        wave++; kills = 0; killsNeededForWave = 5 + wave*2;
        document.getElementById('waveVal').textContent = wave;
        const toSpawn = 3 + wave;
        for(let i=0;i<toSpawn;i++) setTimeout(spawnEnemy, i*250);
    }
}

for(let i=0;i<4;i++) spawnEnemy();

// ----- loot system -----
const LOOT_TYPES = ['ammo', 'health', 'weapon'];
const lootPoints = [
    [10,3], [-10,3], [3,10], [3,-10], [-6,-11], [6,11], [-16,6], [16,-6],
    [0,0], [20,-20], [-20,20], [12,-14]
];
const loot = [];
function spawnLootAt(idx){
    const [x,z] = lootPoints[idx];
    const type = LOOT_TYPES[Math.floor(Math.random()*LOOT_TYPES.length)];
    let color, weaponKey=null;
    if(type==='ammo') color=0xffd23f;
    else if(type==='health') color=0x4caf50;
    else { const keys=['SMG','Shotgun','Sniper']; weaponKey =
        keys[Math.floor(Math.random()*keys.length)]; color = WEAPONS[weaponKey].color; }
    const geo = new THREE.OctahedronGeometry(0.4,0);
    const mat = new THREE.MeshStandardMaterial({color, emissive:color, emissiveIntensity:0.35,
        roughness:0.4});
    const mesh = new THREE.Mesh(geo, mat);
    mesh.position.set(x, 1.1, z);
    mesh.castShadow = true;
    scene.add(mesh);
    loot[idx] = { mesh, x, z, type, weaponKey, active:true, bob: Math.random()*Math.PI*2 };
}
lootPoints.forEach((_,i)=>spawnLootAt(i));

function showPickupToast(text){
    const el = document.getElementById('pickupToast');
    el.textContent = text;
    el.style.opacity = 1;
    el.style.transform = 'translate(-50%,-60%)';
    setTimeout(()=>{ el.style.opacity=0; el.style.transform='translate(-50%,-50%)'; }, 700);
}

function collectLoot(item){
    item.active = false;
    scene.remove(item.mesh);
    if(item.type==='ammo'){
        const w = WEAPONS[currentWeaponKey];
        reserve[currentWeaponKey] = Math.min(w.reserveMax, reserve[currentWeaponKey] +
            Math.round(w.magSize*1.5));
        showPickupToast('+ Ammo');
    } else if(item.type==='health'){
        player.health = Math.min(player.maxHealth, player.health + 35);
        showPickupToast('+ Health');
    } else {
        currentWeaponKey = item.weaponKey;
        reserve[item.weaponKey] = Math.max(reserve[item.weaponKey],
            WEAPONS[item.weaponKey].magSize*2);
        ammo = WEAPONS[item.weaponKey].magSize;
        showPickupToast('Picked up ' + item.weaponKey);
    }
    updateHUD();
    const idx = loot.indexOf(item);
    setTimeout(()=>{ if(idx>=0) spawnLootAt(idx); }, 14000);
}

// ----- collision helper -----
function collidesWithObstacle(x,z,radius){
    for(const o of obstacles){
        if(Math.abs(x-o.x) < o.hw+radius && Math.abs(z-o.z) < o.hd+radius) return true;
    }
}

```

```

return false;
}

function flashDamage(){
    hitFlashEl.style.opacity = 0.55;
    setTimeout(()=>hitFlashEl.style.opacity=0, 150);
}

// ----- HUD -----
function updateHUD(){
    document.getElementById('scoreVal').textContent = score;
    document.getElementById('weaponName').textContent = currentWeaponKey;
    document.getElementById('ammoVal').textContent = ammo;
    document.getElementById('reserveVal').textContent = reserve[currentWeaponKey];
    document.getElementById('healthBarInner').style.width = Math.max(0,player.health) + '%';
    if(!reloading) document.getElementById('reloadHint').textContent = isTouch ? 'Tap reload' : 'R
        to reload';
}
updateHUD();

// ----- minimap -----
const mmCanvas = document.getElementById('minimap');
const mmCtx = mmCanvas.getContext('2d');
function drawMinimap(){
    const S = 130, scale = S/(ARENA*2.2);
    mmCtx.clearRect(0,0,S,S);
    mmCtx.fillStyle = 'rgba(20,30,35,0.6)';
    mmCtx.fillRect(0,0,S,S);
    function toMM(x,z){ return [S/2 + x*scale, S/2 + z*scale]; }
    mmCtx.fillStyle = 'rgba(120,140,130,0.5)';
    obstacles.forEach(o=>{
        const [mx,mz] = toMM(o.x,o.z);
        mmCtx.fillRect(mx-o.hw*scale, mz-o.hd*scale, o.hw*2*scale, o.hd*2*scale);
    });
    loot.forEach(item=>{
        if(!item || !item.active) return;
        const [mx,mz] = toMM(item.x,item.z);
        mmCtx.fillStyle = item.type==='ammo' ? '#ffd23f' : item.type==='health' ? '#4caf50' :
            '#3fa9f5';
        mmCtx.beginPath(); mmCtx.arc(mx,mz,3,0,7); mmCtx.fill();
    });
    mmCtx.fillStyle = '#e74c3c';
    enemies.forEach(en=>{
        if(!en.alive) return;
        const [mx,mz] = toMM(en.mesh.position.x, en.mesh.position.z);
        mmCtx.beginPath(); mmCtx.arc(mx,mz,3,0,7); mmCtx.fill();
    });
    const [px,pz] = toMM(player.pos.x, player.pos.z);
    mmCtx.save();
    mmCtx.translate(px,pz);
    mmCtx.rotate(player.yaw);
    mmCtx.fillStyle = '#ffb020';
    mmCtx.beginPath();
    mmCtx.moveTo(0,-6); mmCtx.lineTo(4,5); mmCtx.lineTo(-4,5); mmCtx.closePath(); mmCtx.fill();
    mmCtx.restore();
}

// ----- game over -----
function gameOver(){
    player.alive = false;
    if(!isTouch) document.exitPointerLock();
    gameActive = false;
    touchControls.classList.remove('active');
    overlay.classList.remove('hidden');
    startBtn.style.display='none';
    const stats = document.getElementById('finalStats');
    stats.style.display='block';
    stats.innerHTML = 'GAME OVER<br>Final Score: '+score+' &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;& Wave Reached: '+wave;
    overlay.querySelector('h1').textContent = 'Defeated';
}

// ----- main loop -----
const clock = new THREE.Clock();
function animate(){
    requestAnimationFrame(animate);
    const dt = Math.min(clock.getDelta(), 0.05);

    if(gameActive && player.alive){
        const forward = new THREE.Vector3(Math.sin(player.yaw), 0, Math.cos(player.yaw)).negate();

```

```

const right = new THREE.Vector3(Math.cos(player.yaw), 0, -Math.sin(player.yaw));
let move = new THREE.Vector3();

if(isTouch){
  if(Math.abs(joyVec.x)>0.08 || Math.abs(joyVec.y)>0.08){
    move.add(forward.clone()).multiplyScalar(-joyVec.y);
    move.add(right.clone()).multiplyScalar(joyVec.x);
  }
} else {
  if(keys['KeyW']) move.add(forward);
  if(keys['KeyS']) move.sub(forward);
  if(keys['KeyD']) move.add(right);
  if(keys['KeyA']) move.sub(right);
}

if(move.lengthSq(>0){
  move.normalize();
  const spd = player.speed * ((!isTouch && (keys['ShiftLeft']||keys['ShiftRight'])) ?
    player.sprintMult : 1);
  const nx = player.pos.x + move.x*spd*dt;
  const nz = player.pos.z + move.z*spd*dt;
  const clampedX = Math.max(-ARENA+1, Math.min(ARENA-1, nx));
  const clampedZ = Math.max(-ARENA+1, Math.min(ARENA-1, nz));
  if(!collidesWithObstacle(clampedX, player.pos.z, 0.5)) player.pos.x = clampedX;
  if(!collidesWithObstacle(player.pos.x, clampedZ, 0.5)) player.pos.z = clampedZ;
}
camera.position.set(player.pos.x, player.height, player.pos.z);
const dir = new THREE.Vector3(
  Math.sin(player.yaw)*Math.cos(player.pitch),
  Math.sin(player.pitch),
  -Math.cos(player.yaw)*Math.cos(player.pitch)
);
camera.lookAt(camera.position.clone().add(dir));

enemies.forEach(en=>{
  if(!en.alive) return;
  const ex=en.mesh.position.x, ez=en.mesh.position.z;
  const dx = player.pos.x - ex, dz = player.pos.z - ez;
  const dist = Math.sqrt(dx*dx+dz*dz);
  if(dist > 1.6){
    const nx = ex + (dx/dist)*en.speed*dt;
    const nz = ez + (dz/dist)*en.speed*dt;
    en.mesh.position.x = nx;
    en.mesh.position.z = nz;
    en.mesh.rotation.y = Math.atan2(dx,dz);
  } else {
    en.attackCd -= dt;
    if(en.attackCd<=0){
      en.attackCd = 0.8;
      player.health -= 8;
      flashDamage();
      updateHUD();
      if(player.health<=0) gameOver();
    }
  }
});

loot.forEach(item=>{
  if(!item || !item.active) return;
  item.bob += dt*2.2;
  item.mesh.position.y = 1.1 + Math.sin(item.bob)*0.12;
  item.mesh.rotation.y += dt*1.4;
  const dx = player.pos.x-item.x, dz = player.pos.z-item.z;
  if(Math.sqrt(dx*dx+dz*dz) < 1.4) collectLoot(item);
});

drawMinimap();
}

renderer.render(scene, camera);
}
animate();
})();

```